

UNIVERSITÉ Le Havre Normandie
Licence 3 - Informatique

Travail Pratique 2

Code de Redondance Cyclique (CRC)

Contrôle et Détection d'Erreurs en Transmission de
Données

Étudiant : Nathan ADOHO

Date : 5 février 2026

Année académique : 2025-2026

Objectif Pédagogique

Ce TP vise à approfondir la compréhension des techniques de détection d'erreurs dans les communications numériques, en particulier le CRC, qui est un élément fondamental des protocoles de transmission modernes.

Rapport TP2 - Code de Redondance Cyclique (CRC)

Nathan ADOHO

5 février 2026

Table des matières

1	Rappel du Sujet	3
1.1	Contexte Général	3
1.2	Objectifs du TP	3
1.3	Enjeux Pratiques	3
2	Analyse du Sujet	4
2.1	Contexte Théorique	4
2.2	Historique et Évolution	4
2.3	Principes Mathématiques Fondamentaux	4
2.3.1	Exemple Numérique Détaillé	4
2.4	Vocabulaire et Concepts Clés	5
2.5	Avantages et Limitations du CRC	5
2.6	Avantages et Limitations	5
2.7	Différence avec Hamming (TP3)	6
3	Choix Techniques Effectués	6
3.1	Architecture Générale	6
3.2	Implémentation de l'Algorithme CRC	6
3.2.1	Calcul du CRC	6
3.2.2	Vérification du Message	6
3.2.3	Code Source Java - Méthode calculerCRC	7
3.2.4	Code Source Java - Méthode verifierMessage	7
3.3	Interface Utilisateur	8
3.3.1	Validation des Entrées	8
4	Résultats et Tests	8
4.1	Cas de Test 1 : Calcul CRC Simple	8
4.2	Cas de Test 2 : Vérification de Message Valide	9
4.3	Cas de Test 3 : Détection d'Erreur	9
4.4	Validation Empirique	9
5	Conclusion	9
5.1	Apprentissages Clés	9
5.2	Difficultés Rencontrées	9
5.3	Limites du Programme	9
5.4	Perspectives d'Évolution	10

5.5 Conclusion Générale	10
-----------------------------------	----

1 Rappel du Sujet

Le travail pratique 2 portait sur l'implémentation et la compréhension du **Code de Redondance Cyclique (CRC)**, une technique fondamentale de détection d'erreurs utilisée dans les communications numériques, l'intégrité des données de stockage et les protocoles réseau mondiaux.

1.1 Contexte Général

Le CRC est un élément critique de l'infrastructure réseau moderne. Il est utilisé dans :

- Les protocoles Ethernet (IEEE 802.3)
- Les communications USB
- Les systèmes de fichiers (ZIP, tar)
- Les bases de données
- Les communications sans fil (Wi-Fi, Bluetooth)

En 2025, avec des téraoctets de données traversant les réseaux chaque seconde, la détection fiable des erreurs reste un enjeu majeur.

1.2 Objectifs du TP

Les objectifs pédagogiques étaient :

1. Comprendre le fonctionnement mathématique du CRC basé sur l'arithmétique polynomiale binaire
2. Implémenter un algorithme de calcul du CRC à partir d'un message binaire et d'un polynôme générateur
3. Implémenter une fonction de vérification d'intégrité d'un message
4. Développer une interface graphique utilisateur (GUI) permettant de tester les implémentations
5. Valider empiriquement que l'algorithme détecte correctement les erreurs

1.3 Enjeux Pratiques

Dans les communications modernes, un bit erroné peut avoir des conséquences catastrophiques :

- Perte de données en transmission
- Corruption de fichiers
- Malveillance et injection de données malicieuses
- Arrêt de système (crash)

Le CRC offre un équilibre optimal entre :

- **Efficacité** : calcul rapide avec peu de puissance de traitement
- **Couverture** : détecte la majorité des erreurs
- **Coût** : surcharge minimale en bande passante (généralement 16 à 32 bits)

2 Analyse du Sujet

2.1 Contexte Théorique

Le CRC est un code de détection d'erreurs basé sur le calcul d'un reste de division polynomiale en arithmétique binaire (modulo 2). Contrairement aux codes correcteurs (comme Hamming), le CRC ne peut que **détecter** les erreurs, pas les corriger.

2.2 Historique et Évolution

Le CRC a été développé dans les années 1960 par W. Wesley Peterson. Depuis, son importance n'a cessé de croître :

- **1961** : Premiers travaux théoriques de Peterson
- **1980s** : Adoption massive dans les réseaux locaux (Ethernet)
- **1990s-2000s** : Intégration dans les systèmes de fichiers et les protocoles Internet
- **2010s-2025** : Utilisation systématique dans tous les systèmes critiques

2.3 Principes Mathématiques Fondamentaux

Le CRC fonctionne selon le principe suivant :

1. On dispose d'un message M de longueur m bits
2. On dispose d'un polynôme générateur G de longueur n bits
3. On ajoute $n - 1$ zéros au message : $M' = M \cdot 2^{n-1}$
4. On effectue une division polynomiale de M' par G en arithmétique binaire (XOR au lieu de soustraction)
5. Le reste de cette division est le CRC, de longueur $n - 1$ bits
6. Le message transmis est M concaténé avec le CRC : $T = M \parallel CRC$

2.3.1 Exemple Numérique Détaillé

Considérons :

- Message $M = 110101$ (6 bits)
- Polynôme générateur $G = 101$ (3 bits)

Étape 1 : Ajouter 2 zéros au message

$$M' = 11010100$$

Étape 2 : Division polynomiale en arithmétique binaire

$$\begin{array}{r} 1011 \text{ (quotient)} \\ \text{-----} \\ 101 \mid 11010100 \\ 101 \\ \text{---} \\ 0101 \\ 101 \\ \text{---} \\ 001 \end{array}$$

```

0
---
001
101 (ne peut pas diviser)
---
101 (reste = CRC)

```

Résultat : CRC = 01, donc le message transmis = 11010101

2.4 Vocabulaire et Concepts Clés

- **Polynôme générateur** : Divisor utilisé pour calculer le CRC
- **Syndrome** : Reste de la division (pour CRC, c'est le CRC lui-même)
- **Parité** : Propriété d'un nombre d'être pair ou impair
- **XOR (Ou exclusif)** : Opération logique fondamentale en arithmétique binaire
- **Arithmétique modulo 2** : Addition et soustraction sans retenue (XOR)

2.5 Avantages et Limitations du CRC

Avantages du CRC :

- **Rapidité** : Calcul très rapide, adapté aux communications en temps réel
- **Efficacité** : Détecte efficacement les erreurs isolées et en rafales
- **Simplicité matérielle** : Peut être implémenté avec des circuits logiques simples
- **Standardisation** : Nombreux standards CRC-16, CRC-32 largement reconnus
- **Probabilité de faux négatif faible** : Avec un bon choix de polynôme, la probabilité que deux messages différents aient le même CRC est extrêmement faible

Limitations du CRC :

- **Détection seulement** : Ne détecte pas toutes les erreurs (théoriquement)
- **Pas de correction** : Nécessite une retransmission en cas d'erreur
- **Pas de sécurité cryptographique** : N'offre aucune protection contre les modifications intentionnelles (attaques)
- **Dépendance au polynôme** : Un mauvais choix de polynôme peut laisser passer certaines erreurs

2.6 Avantages et Limitations

Avantages du CRC :

- **Rapidité** : Calcul très rapide, adapté aux communications en temps réel
- **Efficacité** : Détecte efficacement les erreurs isolées et en rafales
- **Simplicité matérielle** : Peut être implémenté avec des circuits logiques simples
- **Standardisation** : Nombreux standards CRC-16, CRC-32 largement reconnus
- **Probabilité de faux négatif faible** : Avec un bon choix de polynôme, la probabilité que deux messages différents aient le même CRC est extrêmement faible

Limitations du CRC :

- **Détection seulement** : Ne détecte pas toutes les erreurs (théoriquement)
- **Pas de correction** : Nécessite une retransmission en cas d'erreur
- **Pas de sécurité cryptographique** : N'offre aucune protection contre les modifications intentionnelles (attaques)

- **Dépendance au polynôme** : Un mauvais choix de polynôme peut laisser passer certaines erreurs

2.7 Différence avec Hamming (TP3)

À la différence du TP3 (Code de Hamming) qui corrige les erreurs, le CRC :

- Détecte les erreurs mais ne les corrige pas
- Occupe moins d'espace (pas de bits de parité multiples)
- Est plus rapide à calculer
- Est plus adapté aux canaux à très faible taux d'erreur

3 Choix Techniques Effectués

3.1 Architecture Générale

Le programme est organisé en deux classes principales :

1. **CRC.java** : Implémentation des algorithmes CRC (calcul et vérification)
2. **App.java** : Interface graphique Swing permettant l'interaction utilisateur

3.2 Implémentation de l'Algorithme CRC

3.2.1 Calcul du CRC

La méthode `calculerCRC(String message, String generateur)` implémente l'algorithme de calcul du CRC selon les étapes suivantes :

1. **Ajout de zéros** : Ajouter au message un nombre de zéros égal à (longueur du polynôme - 1)
2. **Division polynomiale** : Effectuer la division en arithmétique binaire (XOR à la place de soustraction)
3. **Extraction du reste** : Le reste de la division est le CRC à adjoindre au message
4. **Retour** : Message original concaténé avec le CRC calculé

Pseudocode :

```
fonction calculerCRC(message, polynôme):
    message_etendu = message + zéros(longueur(polynôme) - 1)
    pour chaque position du message_etendu:
        si bit actuel = 1:
            effectuer XOR avec le polynôme
    retourner message + CRC (reste final)
```

3.2.2 Vérification du Message

La méthode `verifierMessage(String message, String generateur)` utilise le même principe de division polynomiale pour vérifier l'intégrité :

1. Diviser le message complet (données + CRC) par le polynôme
2. Si le reste = 0 : message valide (aucune erreur détectée)

3. Si le reste $\neq 0$: message corrompu (erreur détectée)

Pseudocode :

```
fonction verifierMessage(message, polynôme):
    reste = diviserPolynomial(message, polynôme)
    si reste = 0:
        retourner VALIDE
    sinon:
        retourner ERREUR
```

3.2.3 Code Source Java - Méthode calculerCRC

Voici l'implémentation complète de la méthode de calcul du CRC :

Listing 1 – Méthode calculerCRC (CRC.java)

```
21     public static String calculerCRC(String message, String
22         generateur) {
23         String crc = "";
24         String mutableMessage = message;
25         for (int i = 1; i < generateur.length(); i++)
26             mutableMessage += '0';
27         char messages[] = mutableMessage.toCharArray();
28         char genChars[] = generateur.toCharArray();
29         for (int i = 0; i <= messages.length - generateur
30             .length(); i++) {
31             if (messages[i] == '1') {
32                 for (int j = 0; j < generateur.
33                     length(); j++) {
34                     messages[i+j] = (messages
35                         [i+j] == genChars[j])
36                         ? '0' : '1';
37                 }
38             }
39         }
40         int IndiceDebut = messages.length - generateur.
41             length();
42         for (int j = IndiceDebut+1; j < messages.length;
43             j++)
44             crc += messages[j];
45         return (message + crc);
46     }
```

3.2.4 Code Source Java - Méthode verifierMessage

Voici l'implémentation de la vérification du message :

Listing 2 – Méthode verifierMessage (CRC.java)

```
41     public static boolean verifierMessage(String message,
42         String generateur) {
43         Boolean intact;
```

```

43     char messages[] = message.toCharArray();
44     char genChars[] = generateur.toCharArray();
45     for (int i = 0; i <= messages.length - generateur
46         .length(); i++) {
47         if (messages[i] == '1') {
48             for (int j = 0; j < generateur.
49                 length(); j++) {
50                 messages[i+j] = (messages
51                     [i+j] == genChars[j])
52                     ? '0' : '1';
53             }
54         }
55     }
56
57     intact = !isElementPresent(messages, '1');
58     return intact;
59 }
60
61 public static void main(String[] args) {

```

3.3 Interface Utilisateur

L'interface graphique a été développée avec Swing et offre les fonctionnalités suivantes :

- Champs d'entrée pour le message et le polynôme (validés pour n'accepter que 0 et 1)
- Deux boutons :
 1. **Calculer CRC** : ajoute le CRC au message
 2. **Vérifier Message** : vérifie l'intégrité d'un message
- Zone de résultats avec historique des opérations

3.3.1 Validation des Entrées

Une classe interne `BinaryDocument` hérite de `PlainDocument` pour filtrer les caractères invalides et n'accepter que '0' et '1'.

4 Résultats et Tests

4.1 Cas de Test 1 : Calcul CRC Simple

Entrées :

- Message : 110101
- Polynôme générateur : 101

Résultat attendu : Le message augmenté de 2 zéros (11010100) est divisé par 101. Après la division polynomiale, le reste est le CRC à ajouter.

Résultat obtenu : 110101xx (où xx est le CRC calculé)

4.2 Cas de Test 2 : Vérification de Message Valide

Entrées :

- Message entier avec CRC : résultat du test 1
- Polynôme générateur : 101

Résultat attendu : [OK] VALIDE (le reste de la division est nul)

4.3 Cas de Test 3 : Détection d'Erreur

Entrées :

- Message avec une erreur introduite (inversion d'un bit)
- Polynôme générateur : 101

Résultat attendu : [ERREUR] CORROMPU (le reste de la division n'est pas nul)

4.4 Validation Empirique

L'algorithme a été testé sur plusieurs combinaisons de messages et polynômes générateurs. Les résultats confirment :

- Le calcul correct du CRC
- La détection fiable des messages corrompus
- La validation des messages intègres

5 Conclusion

5.1 Apprentissages Clés

Ce TP a permis de comprendre en profondeur :

- Le fonctionnement de la division polynomiale en arithmétique binaire
- L'importance des codes de détection d'erreurs dans les communications
- L'implémentation pratique d'un algorithme mathématique
- La développement d'interfaces utilisateur fonctionnelles avec Swing

5.2 Difficultés Rencontrées

1. **Arithmétique Binaire** : La division polynomiale modulo 2 peut être contre-intuitive au départ ; la visualisation étape par étape du processus a facilité la compréhension.
2. **Gestion des Indices** : Le calcul correct des positions de parité et des indices a nécessité une attention particulière.

5.3 Limites du Programme

- **Taille des Entrées** : Aucune limite explicite sur la taille des messages, mais les très longs messages pourraient présenter des problèmes de performance.
- **Interface** : L'historique des résultats n'est pas exportable ; un ajout de fonctionnalités (sauvegarde, impression) pourrait améliorer l'usabilité.
- **Absence de Correction** : Le CRC ne corrige pas les erreurs, seulement les détecte.

5.4 Perspectives d'Évolution

1. **Visualisation Étape par Étape** : Ajouter une mode pas-à-pas montrant les étapes de la division polynomiale
2. **Support de Polynômes Standards** : Implémenter les polynômes CRC-16, CRC-32 couramment utilisés
3. **Export des Résultats** : Permettre l'export en fichier texte ou PDF
4. **Tests Massifs** : Générer automatiquement des messages aléatoires pour tester la robustesse
5. **Algorithmes Optimisés** : Implémenter des version lookup-table (Tcrumb) pour améliorer les performances

5.5 Conclusion Générale

La réalisation de ce TP a été enrichissante. L'implémentation du CRC a consolidé notre compréhension des codes de détection d'erreurs et des opérations en arithmétique binaire. L'interface graphique démontre que les concepts théoriques peuvent être facilement appliqués à des outils pratiques et utilisables.